

Кросплатформне програмування

13

Стандартні колекції



Стандартні колекції

Про що йтиметься:

- Інтерфейси колекцій
- Інтерфейси Collection , List, Set та SortedSet
- Класи Collection: ArrayList, Vector, LinkedList, HashSet, TreeSet
- Ітератори
- Компаратори
- Алгоритми



Знайомство з колекціями

Колекції - набір спеціальних утиліт для роботи з групами об'єктів

Основні операції, що підтримуються для груп об'єктів:

- Додавання елемента (об'єкта) до групи
- Видалення елемента (об'єкта) з групи
- Перебір елементів (об'єктів) у групі

Механізми реалізації колекцій:

- Набір інтерфейсів, які визначають стандарти роботи з колекціями
- Набір класів, які реалізують основні інтерфейси, призначені для роботи з колекціями
- Набір алгоритмів (спеціальні методи) для роботи з колекціями
- Ітератори – спеціальні об'єкти, призначені для реалізації доступу до елементів колекцій



Основні інтерфейси

Collection - основний інтерфейс, містить базові методи, які повинна мати будь-яка колекція

List - розширює інтерфейс Collection. Інтерфейс для реалізації колекції типу списку: упорядкованої послідовності елементів, до яких можна отримати доступ за індексом

Set - розширює інтерфейс Collection. Інтерфейс для реалізації множини: набору унікальних елементів, порядок слідування яких не важливий

SortedSet - розширює інтерфейс Set. Інтерфейс призначений для реалізації впорядкованих множин

Queue - розширює інтерфейс Collection. Інтерфейс для реалізації черги: колекції, в якій елементи додаються в один кінець, а зчитуються з іншого кінця

Map - інтерфейс для реалізації колекції, в якій використовуються ключі та значення



Додаткові інтерфейси

Comparator - інтерфейс для визначення способу порівняння об'єктів в колекції

Iterator – інтерфейс для створення ітераторів для роботи з колекціями (циклічний перебір колекції з можливістю додавання та видалення елементів)

ListIterator - розширює інтерфейс Iterator. Інтерфейс для доступу до елементів колекції типу списку. Дозволяє виконувати двонаправлений обхід списку та модифікацію його елементів



Інтерфейс Collection

Містить оголошення основних методів, загальних для всіх колекцій

Метод	Описання
<code>boolean add(Object obj)</code>	Додає об'єкт <code>obj</code> до колекції. Результат дорівнює <code>true</code> якщо об'єкт доданий, та <code>false</code> якщо об'єкт вже є елементом колекції
<code>boolean addAll(Collection c)</code>	Додає у вихідну колекцію (з якої викликається метод) всі елементи колекції <code>c</code> . Результатом є <code>true</code> якщо операція пройшла вдало, і <code>false</code> в іншому випадку
<code>clear()</code>	Видаляє всі елементи з колекції
<code>boolean contains(Object obj)</code>	Повертає <code>true</code> якщо елемент <code>obj</code> міститься в колекції, та <code>false</code> якщо ні
<code>boolean containsAll(Collection c)</code>	Повертає <code>true</code> якщо всі елементи колекції <code>c</code> містяться у вихідній колекції (з якої викликається метод), та <code>false</code> якщо ні
<code>boolean equals(Object obj)</code>	Перевірка щодо рівності об'єкта колекції та об'єкта, переданого аргументом (дві однакові колекції)
<code>int hashCode()</code>	Повертає хеш-код об'єкта колекції
<code>boolean isEmpty()</code>	Перевірка колекції щодо того, що вона порожня
<code>Iterator iterator()</code>	Повертає ітератор для колекції



Інтерфейс Collection

Метод	Описання
<code>boolean remove(Object obj)</code>	Видаляє елемент <code>obj</code> з колекції виклику методу. Результатом є <code>true</code> якщо елемент видалено з колекції, та <code>false</code> в іншому випадку
<code>boolean removeAll(Collection c)</code>	Видаляє всі елементи колекції <code>c</code> із колекції виклику методу. Результатом є <code>true</code> якщо елементи видалені, та <code>false</code> в іншому випадку
<code>boolean retainAll(Collection c)</code>	Видаляє всі елементи з колекції, з якої викликається метод, за винятком елементів з колекції <code>c</code> . Результатом є <code>true</code> якщо елементи видалені, та <code>false</code> в іншому випадку
<code>int size()</code>	Повертає розмір колекції – кількість елементів, що містяться в колекції
<code>Object[] toArray()</code>	Повертає масив, який містить всі елементи, що зберігаються в колекції, з якої викликається метод. Елементи масиву є копіями елементів колекції



Інтерфейс List

Інтерфейс **List** розширює інтерфейс **Collection**

Елемент можна вставляти та вилучати за індексом (індексація починається з нуля)

Може містити елементи, що дублюються (збігаються)

Метод	Описання
<code>void add(int index, Object obj)</code>	Вставляє об'єкт <code>obj</code> до списку виклику у позицію, що визначається індексом <code>index</code> . Існуючі елементи зміщуються
<code>boolean addAll(int index, Collection c)</code>	Додає (починаючи з позиції <code>index</code>) у вихідний список (з якого викликається метод) всі елементи колекції <code>c</code> . Існуючі елементи зміщуються. Результатом є <code>true</code> якщо список, з якого викликається метод, змінюється, і <code>false</code> в іншому випадку
<code>Object get(int index)</code>	Повертає об'єкт, що зберігається у позиції з індексом <code>index</code>
<code>int indexOf(Object obj)</code>	Повертає індекс першого екземпляра об'єкта <code>obj</code> у списку. Якщо такого об'єкта немає, повертається значення <code>-1</code>
<code>int lastIndexOf(Object obj)</code>	Повертає індекс останнього екземпляра об'єкта <code>obj</code> у списку. Якщо такого об'єкта немає, повертається значення <code>-1</code>



Інтерфейс List

Метод	Описання
<code>ListIterator listIterator()</code>	Повертає ітератор, встановлений на початок списку
<code>ListIterator listIterator(int index)</code>	Повертає ітератор, встановлений на елемент із індексом <code>index</code>
<code>Object remove(int index)</code>	Видаляє зі списку елемент з індексом <code>index</code> . Результатом повертається посилання на віддалений зі списку об'єкт
<code>Object set(int index, Object obj)</code>	Вставляє об'єкт <code>obj</code> у позицію з індексом <code>int</code> . Результатом повертається посилання на об'єкт
<code>List subList(int start, int end)</code>	Повертає список з елементів у вихідному списку (з якого викликається метод) з індексами від <code>start</code> до <code>end-1</code> включно

Інтерфейс Set

Інтерфейс **Set** розширює інтерфейс **Collection**

Не допускає дублювання елементів - при додаванні елемента методом **add()** якщо такий елемент є в колекції, новий не додається (метод поверне значення **false**)

Власних методів не визначає



Інтерфейс SortedSet

Інтерфейс **SortedSet** розширює інтерфейс **Set**
Визначає відсортовану в порядку зростання множину (колекцію)

Метод	Описання
<code>Comparator comparator()</code>	Повертає компаратор для впорядкованої множини. Якщо для множини використовується природне (вбудоване) впорядкування, результатом повертається порожнє посилання <code>null</code>
<code>Object first()</code>	Повертає перший елемент впорядкованої множини
<code>SortedSet headSet(Object obj)</code>	Повертається впорядкована множина з елементів, які менші за об'єкт <code>obj</code>
<code>Object last()</code>	Повертає останній елемент у впорядкованому списку
<code>SortedSet subSet(Object start, Object end)</code>	Повертає список з елементів у вихідному списку (з якого викликається метод), які знаходяться між об'єктами <code>start</code> (включаючи) та <code>end</code> (не включаючи)
<code>SortedSet tailSet(Object obj)</code>	Повертається впорядкована множина з елементів, які більші або рівні за об'єкт <code>obj</code>



Класи, що реалізують Collection

Деякі класи абстрактні, деякі – ні

Клас	Опис
AbstractCollection	Реалізує більшу частину інтерфейсу Collection
AbstractList	Розширює клас AbstractCollection і реалізує більшу частину інтерфейсу List
AbstractSequentialList	Розширює клас AbstractList для реалізації колекції, яка використовує послідовний, а не довільний доступ до елементів
LinkedList	Реалізує зв'язний список, розширюючи клас AbstractList
ArrayList	Реалізує динамічний масив, розширюючи клас AbstractSequentialList
AbstractSet	Розширює клас AbstractCollection і реалізує більшу частину інтерфейсу Set
HashSet	Розширює клас AbstractSet для реалізації хеш-таблиць
TreeSet	Реалізує набір, що зберігається у вигляді дерева (набір деревоподібний). Розширює клас AbstractSet

Для роботи з колекціями також використовуються класи **Vector**, **Stack** і **Hashtable**



Клас ArrayList

- Розширює клас **AbstractList** і реалізує інтерфейс **List**
- Підтримує динамічні масиви, які можуть змінювати розмір під час виконання програми
- Список створюється з початковим розміром. При перевищенні розміру список автоматично розширюється. При видаленні об'єктів розмір може зменшуватись
- Динамічний масив можна реалізувати за допомогою класу **Vector**

Конструктор	Описання
<code>ArrayList()</code>	Створюється порожній список
<code>ArrayList(Collection c)</code>	Створення списку на основі колекції
<code>ArrayList(int capacity)</code>	Створення списку заданого розміру (ємності)



Використання класу ArrayList

```
// Імпорт контейнерного класу:
import java.util.ArrayList;
class Demo{
    //Статичний метод для відображення вмісту контейнера:
    static void show(ArrayList lst){
        for(int k=0;k<lst.size();k++){
            System.out.print("|"+lst.get(k)+" ");
        }
        System.out.println ("|");
    }
    // Головний метод:
    public static void main(String[] args){
        // Створення пустого списку:
        ArrayList<Integer> nums=new ArrayList<>();
        // Додавання елементів до списку:
        nums.add(100);
        nums.add(200);
        nums.add(300);
        // Відображення вмісту списку:
        show(nums);
        // Додавання елементів до списку:
        nums.add(0,-1);
        nums.add(2,0);
        // Відображення вмісту списку:
        show(nums);
        // Продовження на наступному слайді!
```



Використання класу ArrayList

```
// Очищення списку:
nums.clear();
// Якщо список порожній:
if(nums.isEmpty()) nums.add(123);
// Відображення вмісту списку:
show(nums);
// Створення списку зазначеної ємності:
ArrayList<Character> chars=new ArrayList<>(10);
// Додавання елементів до списку:
chars.add('a');
chars.add('b');
chars.add(0,'z');
// Відображення вмісту списку (неявний виклик методу toString()):
System.out.println(chars);
// Перетворення списку на масив:
Object[] myarray=chars.toArray();
// Поелементне відображення елементів масиву:
for(int i=0;i<myarray.length;i++){
    System.out.print((Character)myarray[i]+" ");
}
System.out.println();
}
```

Результат

```
| 100 | 200 | 300 |
| -1 | 100 | 0 | 200 | 300 |
| 123 |
[z, a, b]
zab
```

Клас Vector

- Клас **Vector** подібний до класу **ArrayList**: успадковує клас **AbstractList** і реалізує інтерфейс **List**
- Підтримує динамічні масиви, які можуть змінювати розмір під час виконання програми
- Список створюється з початковим розміром. Коли розмір вичерпано, він збільшується. При створенні контейнера можна вказати розмір добавки до розміру контейнера. Якщо "добавку" не вказати, то щоразу розмір подвоюється

Конструктор	Описання
<code>Vector()</code>	Створюється порожній вектор
<code>Vector(int size)</code>	Створення вектора вказаного розміру
<code>Vector(int size, ind delta)</code>	Створення вектора вказаного розміру та визначення добавки для зміни розміру
<code>Vector(Collection c)</code>	Створення вектора на основі колекції



Використання класу Vector

```
// Імпорт контейнерного класу:
import java.util.Vector;
class Demo{
    public static void main(String[] args){
        // Створення пустого вектора:
        Vector<Integer> nums=new Vector<>();
        // Додавання елементів у вектор:
        for(int i=1;i<=3;i++){
            nums.addElement(i *100);
        }
        // Відображення вмісту вектора:
        System.out.println(nums);
        // Додавання елементів у вектор:
        nums.add(0,-1);
        nums.add(2,0);
        // Відображення вмісту вектора:
        System.out.println(nums);
        // Зміна розміру вектора:
        nums.setSize(nums.size()-2);
        // Відображення вмісту вектора:
        System.out.println(nums);
    }
}
```

Результат

```
[100, 200, 300]
[-1, 100, 0, 200, 300]
[-1, 100, 0]
```




Клас LinkedList

- Розширює клас **AbstractSequentialList** і реалізує інтерфейс **List**
- Підтримує зв'язний список
- Список створюється пустим або на основі деякої колекції
- До списку можна додавати елементи, і можна видалити елемент зі списку

Конструктор	Описання
<code>LinkedList()</code>	Створюється порожній список
<code>LinkedList(Collection c)</code>	Створення списку на основі колекції

Додаткові методи

Метод	Описання
<code>addFirst(Object obj)</code>	Додається перший елемент
<code>addLast(Object obj)</code>	Додається останній елемент
<code>getFirst()</code>	Отримати перший елемент
<code>getLast()</code>	Отримати останній елемент
<code>removeFirst()</code>	Видалити перший елемент
<code>removeLast()</code>	Видалити останній елемент

Основна відмінність класів **ArrayList** та **LinkedList** - у способі реалізації списків.

- Контейнер на основі класу **ArrayList** створюється з певним розміром "із запасом". Розмір контейнера змінюється за необхідності.
- Список на основі класу **LinkedList** містить ланцюжок елементів - рівно стільки, скільки елементів додано до списку



Використання класу LinkedList

```
// Імпорт контейнерного класу:
import java.util.LinkedList;
class Demo{
    public static void main(String[] args){
        // Створення пустого списку:
        LinkedList<Integer> nums=new LinkedList<>();
        // Додавання елементів до списку:
        nums.add(100);
        nums.add(0,200);
        nums.addLast(300);
        System.out.println(nums); // Відображення вмісту списку
        // Додавання та видалення елементів:
        nums.removeFirst();
        nums.add(0,400);
        nums.removeLast();
        nums.addLast(500);
        System.out.println(nums); // Відображення вмісту списку
        // Перетворення списку на масив:
        Object[] myarray=nums.toArray();
        // Поелементне відображення елементів масиву:
        for(int i=0;i<myarray.length;i++){
            System.out.print((Integer)myarray[i]+" ");
        }
        System.out.println();
    }
}
```

Результат

```
[200, 100, 300]
[400, 100, 500]
400 100 500
```



Клас HashSet

- Розширює клас **AbstractSet** і реалізує інтерфейс **Set**
- Підтримує колекцію на основі хеш-таблиці: інформаційне значення (елемент) використовується для формування унікального хеш-коду. Цей хеш-код використовується як "індекс" елемента

Конструктор	Опис
<code>HashSet()</code>	Створюється пуста колекція
<code>HashSet(Collection c)</code>	Створення колекції на основі колекції
<code>HashSet(int capacity)</code>	Створюється колекція вказаної ємності
<code>HashSet(int capacity, float fillRatio)</code>	Створюється колекція зазначеної ємності та рівня заповнення (параметр від 0.0 до 1.0)

Важливі методи

Метод	Описання
<code>add(Object obj)</code>	Додається елемент до колекції
<code>contains(Object obj)</code>	Перевірка наявності елемента в колекції
<code>remove(Object obj)</code>	Видалення елемента з колекції
<code>size()</code>	Розмір колекції



Використання класу HashSet

```
// Імпорт контейнерного класу:
import java.util.HashSet;
class Demo{
    public static void main(String[] args){
        // Створення порожньої колекції:
        HashSet<String> objs=new HashSet<>();
        // Додавання елементів до колекції:
        objs.add("ПЕРШИЙ");
        objs.add ("ДРУГИЙ");
        objs.add ("ТРЕТІЙ");
        // Відображення вмісту колекції:
        System.out.println(objs);
        // Видалення елемента з колекції:
        objs.remove("ПЕРШИЙ");
        // Додавання елемента в колекцію:
        objs.add("ЧЕТВЕРТИЙ");
        // Кількість елементів у колекції:
        System.out.println("У колекції "+objs.size()+" елементів:\n"+objs);
        // Наявність/відсутність елементів у колекції:
        if(!objs.contains("ПЕРШИЙ"))
            System.out.println("Елемента ПЕРШИЙ в колекції немає");
        if(objs.contains("ЧЕТВЕРТИЙ"))
            System.out.println(" Елемент ЧЕТВЕРТИЙ в колекції є");
    }
}
```

Результат

[ПЕРШИЙ, ТРЕТІЙ, ДРУГИЙ]
У колекції 3 елементів:
[ЧЕТВЕРТИЙ, ТРЕТІЙ, ДРУГИЙ]
Елемента ПЕРШИЙ в колекції немає
Елемент ЧЕТВЕРТИЙ в колекції є



Використання класу HashSet

```
// Імпорт контейнерного класу:
import java.util.HashSet;
// Клас:
class MyClass{
    // Поле:
    int num;
    // Конструктор:
    MyClass(int n){
        num=n;
    }
    // Переозначення методу toString():
    public String toString(){
        return "Об'єкт класу MyClass: "+num;
    }
}
// Головний клас:
class Demo{
    public static void main(String[] args){
        // Створення порожньої колекції:
        HashSet objs=new HashSet();
        // Додавання елементів до колекції:
        objs.add("ПЕРШИЙ");
        objs.add(200);
        objs.add('R');
        objs.add(new MyClass(123));
        // Продовження на наступному слайді!
```



Використання класу HashSet

```
// Відображення вмісту колекції:
System.out.println(objs);
// Видалення елемента з колекції:
objs.remove("ПЕРШИЙ");
// Додавання елемента в колекцію:
objs.add("ЧЕТВЕРТИЙ");
// Кількість елементів у колекції:
System.out.println("У колекції "+objs.size()+" елементів:\ n"+objs);
// Наявність/відсутність елементів у колекції:
if(!objs.contains("ПЕРШИЙ"))
    System.out.println("Елемента ПЕРШИЙ в колекції немає");
if(objs.contains("ЧЕТВЕРТИЙ"))
    System.out.println("Елемент ЧЕТВЕРТИЙ в колекції є");
}
```

Результат

```
[R, ПЕРШИЙ, Об'єкт класу MyClass: 123, 200]
У колекції 4 елементів:
[R, Об'єкт класу MyClass: 123, 200, ЧЕТВЕРТИЙ]
Елемента ПЕРШИЙ в колекції немає
Елемент ЧЕТВЕРТИЙ в колекції є
```



Клас TreeSet

- Реалізує інтерфейс **Set**
- Для зберігання даних використовується деревоподібна структура
- Підходить для зберігання великих наборів сортованої інформації

Конструктор	Описання
<code>TreeSet()</code>	Створюється пуста колекція
<code>TreeSet(Collection c)</code>	Створення колекції на основі колекції
<code>TreeSet(Comparator cmp)</code>	Створюється колекція вказаної ємності
<code>TreeSet(SortedSet st)</code>	Створюється колекція на основі сортованого списку

Важливі методи

Метод	Описання
<code>add(Object obj)</code>	Додається елемент до колекції
<code>contains(Object obj)</code>	Перевірка наявності елемента в колекції
<code>remove(Object obj)</code>	Видалення елемента з колекції
<code>size()</code>	Розмір колекції



Використання класу TreeSet

```
// Імпорт контейнерного класу:
import java.util.TreeSet;
class Demo{
    public static void main(String[] args){
        // Створення пустої колекції:
        TreeSet<String> objs=new TreeSet<>();
        // Додавання елементів до колекції:
        objs.add("ПЕРШИЙ");
        objs.add("ДРУГИЙ");
        objs.add("ТРЕТІЙ");
        // Відображення вмісту колекції:
        System.out.println(objs);
        // Видалення елемента з колекції:
        objs.remove("ПЕРШИЙ");
        // Додавання елемента в колекцію:
        objs.add("ЧЕТВЕРТИЙ");
        // Кількість елементів у колекції:
        System.out.println("У колекції "+objs.size()+" елементів:\ n"+objs);
        // Наявність/відсутність елементів у колекції:
        if(!objs.contains("ПЕРШИЙ"))
            System.out.println("Елемента ПЕРШИЙ в колекції немає");
        if( objs.contains("ЧЕТВЕРТИЙ"))
            System.out.println("Елемент ЧЕТВЕРТИЙ в колекції є");
    }
}
```

Результат

[ДРУГИЙ, ПЕРШИЙ, ТРЕТІЙ]

У колекції 3 елементів:

[ДРУГИЙ, ТРЕТІЙ, ЧЕТВЕРТИЙ]

Елемента ПЕРШИЙ в колекції немає

Елемент ЧЕТВЕРТИЙ в колекції є



Ітератор

- Ітератор – це спеціальний об'єкт, який використовується для перебору елементів у колекції
- Ітератор створюється на основі класу, що реалізує інтерфейс **Iterator** або **ListIterator**
- Інтерфейс **Iterator** дозволяє циклічно пройти колекцію, отримати доступ до елементів або видаляти елементи
- Інтерфейс **ListIterator** розширює **Iterator**, забезпечуючи двонаправлений обхід колекції та модифікацію її елементів

Методи інтерфейсу **Iterator**

Метод	Описання
<code>boolean hasNext()</code>	Повертає <code>true</code> якщо в колекції є наступний елемент. Інакше повертає <code>false</code>
<code>Object next()</code>	Повертає посилання на наступний елемент колекції. Якщо наступного елемента немає, викидається виняток класу <code>NoSuchElementException</code>
<code>void remove()</code>	Видаляє поточний елемент. Викидає виняток класу <code>IllegalStateException</code> (якщо перед викликом <code>remove()</code> не був викликаний метод <code>next()</code>)



Ітератор

Методи інтерфейсу `ListIterator`

Метод	Описання
<code>add(Object obj)</code>	Вставляє <code>obj</code> до списку перед елементом, який буде повернутий за наступного виклику методом <code>next()</code>
<code>boolean hasNext()</code>	Повертає <code>true</code> якщо в колекції є наступний елемент. Інакше повертає <code>false</code>
<code>boolean hasPrevious()</code>	Повертає <code>true</code> якщо в колекції є попередній елемент. Інакше повертає <code>false</code>
<code>Object next()</code>	Повертає посилання на наступний елемент колекції. Якщо наступного елемента немає, викидається виняток класу <code>NoSuchElementException</code>
<code>int nextIndex()</code>	Повертає індекс наступного елемента в списку. Якщо наступного елемента немає, повертається розмір списку
<code>Object previous()</code>	Повертає посилання на попередній елемент колекції. Якщо попереднього елемента немає, викидається виняток класу <code>NoSuchElementException</code>
<code>int previousIndex()</code>	Повертає індекс попереднього елемента у списку. Якщо такого елемента немає, повертається <code>-1</code>
<code>void remove()</code>	Видаляє поточний елемент. Викидає виняток класу <code>IllegalStateException</code> (якщо перед викликом <code>remove()</code> не був викликаний метод <code>next()</code>)
<code>void set(Object obj)</code>	Призначає <code>obj</code> на поточний елемент (елемент, посилання на який отримано викликом методу <code>next()</code> або <code>previous()</code>)



Ітератор

Як використовується ітератор

- У кожному колекційному класі (реалізує інтерфейс **Collection**) є **iterator()**, який повертає посилання на об'єкт-ітератор, встановлений на початок колекції
- Для доступу до чергового елемента колекції використовують метод **next()**
- Метод **next()** можна викликати, якщо метод **hasNext()** повертає значення **true**
- Колекції, створені на основі класу, що реалізує інтерфейс **List**, мають метод **listIterator()**, який повертає посилання на об'єкт-ітератор, що реалізує інтерфейс **ListIterator**



Використання ітератора

```
import java.util.HashSet;
import java.util.Iterator;
import java.util.NoSuchElementException;
class Demo{
    public static void main(String[] args){
        HashSet<String> cls=new HashSet<>();
        cls.add("Червоний");
        cls.add("Жовтий");
        cls.add("Зелений");
        Iterator<String> pnt; // Змінна для запису посилання на ітератор
        pnt=cls.iterator(); // Отримання посилання на ітератор
        while(pnt.hasNext()){
            System.out.println(pnt.next()); // Доступ до наступного елемента
        }
        pnt.remove(); // Видалення елемента через ітератор
        pnt=cls.iterator(); // Посилання на ітератор
        System.out.println("Після видалення елемента");
        try{
            while(true){
                System.out.println(pnt.next()); // Доступ до елемента через ітератор
            }
        } catch(NoSuchElementException e){ // Обробка виключення
            System.out.println("Елементів більше немає!");
        }
    }
}
```

Результат

Зелений

Жовтий

Червоний

Після видалення елемента

Зелений

Жовтий

Елементів більше немає!



Використання ітератора

```
import java.util.ArrayList;
import java.util.ListIterator;
class Demo{
    public static void main(String[] args){
        ArrayList<Integer> nums=new ArrayList<>();
        int k=0;
        nums.add(100);
        nums.add(200);
        nums.add(300);
        ListIterator<Integer> pnt=nums.listIterator (); // Ітератор
        while(pnt.hasNext()){
            System.out.print(pnt.next()+" "); // Прямий перебір
        }
        System.out.println();
        pnt.set(400); // Зміна елемента
        while(pnt.hasPrevious()){
            if(k%2==0) pnt.add(k+1); // Додавання елемента
            k++;
            System.out.print(pnt.previous()+" "); // Зворотний перебір
        }
        System.out.println();
        for(k=0;k<nums.size();k++){
            System.out.print(nums.get(k)+" "); // Перевірка вмісту
        }
        System.out.println();
    }
}
```

Результат

```
100 200 300
1 400 3 200 5 100
100 5 200 3 400 1
```



Компаратор

- Компаратор - спеціальний об'єкт, який використовується для порівняння елементів колекції для їхнього впорядкування
- Компаратор створюється на основі класу, що реалізує інтерфейс **Comparator**

Методи інтерфейсу Comparator

Метод	Описання
<code>int compare(Object a, Object b)</code>	Метод порівняння об'єктів <code>a</code> та <code>b</code> . Якщо метод повертає <code>0</code> , то об'єкти рівні. При позитивному результаті об'єкт <code>a</code> більше об'єкта <code>b</code> . При негативному результаті об'єкт <code>b</code> більше об'єкта <code>a</code> .
<code>boolean equals(Object obj)</code>	Метод використовується для порівняння компараторів: виконується перевірка, чи дорівнює переданий аргументом компараторний об'єкт тому компараторному об'єкту, з якого викликається метод. Метод зазвичай не використовується і його можна не визначати



Використання компаратора

```
import java.util.TreeSet;
import java.util.ArrayList;
import java.util.Comparator;
class MC implements Comparator{ // Клас компаратора
    public int compare(Object a,Object b){
        int x,y;
        x=(Integer)a/10%10;
        y=(Integer)b/10%10;
        return x-y;
    }
}
class Demo{
    public static void main(String[] args){
        ArrayList<Integer> nums=new ArrayList<>();
        int[] n={350,140,530,220,410};
        // int [] n={300,100,500,200,400};
        for(int k=0;k<n.length;k++){
            nums.add(n[k]);
        }
        System.out.println(nums);
        TreeSet<Integer> A=new TreeSet<>(nums);
        System.out.println(A);
        // Використання компаратора:
        TreeSet<Integer> B=new TreeSet<>(new MC());
        B.addAll(nums);
        System.out.println(B);
    }
}
```

Результат

```
[350, 140, 530, 220, 410]
[140, 220, 350, 410, 530]
[410, 220, 530, 140, 350]
```

Результат

```
[300, 100, 500, 200, 400]
[100, 200, 300, 400, 500]
[300]
```



Алгоритми колекцій

Деякі статичні методи класу Collections

Метод	Описання
<code>int binarySearch(List L, Object V, Comparator C)</code>	Пошук об'єкта <code>V</code> зі списку <code>L</code> , упорядкованого за допомогою компаратора <code>C</code> . Повертається позиція об'єкта у списку або <code>-1</code> якщо значення не знайдено
<code>void copy(List L1, List L2)</code>	Копіює елементи <code>L2</code> в <code>L1</code>
<code>Object max(Collection L, Comparator C)</code>	Повертає максимальний елемент у колекції <code>L</code> використовуючи для порівняння елементів компаратор <code>C</code>
<code>Object min(Collection L, Comparator C)</code>	Повертає мінімальний елемент у колекції <code>L</code> використовуючи для порівняння елементів компаратор <code>C</code>
<code>void reverse(List L)</code>	Реверсує послідовність зі списку <code>L</code>
<code>Comparator reverseOrder ()</code>	Повертає компаратор, зворотний до стандартного (компаратор, який реверсує результат порівняння об'єктів).
<code>void sort(List L, Comparator C)</code>	Сортує елементи списку <code>L</code> за правилами компаратора <code>C</code>



Використання алгоритмів

```
import java.util.Collections;
import java.util.Comparator;
import java.util.ArrayList;
// Клас для об'єктів зі списку:
class MyClass{
    private int code;
    private String name;
    MyClass(String name,int code){
        this.name=name;
        this.code=code;
    }
    int get(){
        return code;
    }
    public String toString(){
        return name+"-"+code;
    }
}
//Клас компаратора:
class MC implements Comparator{
    public int compare(Object a,Object b){
        return ((MyClass)a).get()-((MyClass)b).get();
    }
}
// Продовження на наступному слайді!
```



Використання алгоритмів

```
class Demo{
    public static void main(String[] args){
        int[] nums={12,5,17,8,21,25,9};           // Числовий та текстовий масиви
        String[] txt={"ПЕРШИЙ", "ДРУГИЙ", "ТРЕТИЙ", "ЧЕТВЕРТИЙ",
                     "П'ЯТИЙ", "ШОСТИЙ", "СЬОМИЙ"};

        ArrayList<Integer> al=new ArrayList<>();    // Числовий список
        for(int i=0;i<nums.length;i++) al.add(nums[i]); // Заповнення списку
        System.out.println(al);                    // Відображення списку
        Collections.sort(al);                      // Сортування списку
        System.out.println(al);                    // Відображення списку
        Collections.reverse(al);                   // Зворотній порядок
        System.out.println(al);                    // Відображення списку
        for(int k=1;k<=3;k++) al.add(2*k,k*100);    // Додавання елементів
        System.out.println(al);                    // Відображення списку
        Collections.sort(al,Collections.reverseOrder()); // Сортування списку
        System.out.println(al);                    // Відображення списку
        ArrayList<MyClass> ml=new ArrayList<>();    // Список об'єктів
        // Додавання об'єктів до списку:
        for(int k=0;k<nums.length;k++) ml.add(new MyClass(txt[k],nums[k]));
        System.out.println(ml);                    // Відображення списку
        MC cmp=new MC();                          // Компаратор
        System.out.println("Найбільше значення: "+Collections.max(ml,cmp));
        System.out.println("Найменше значення: "+Collections.min(ml,cmp));
        Collections.sort(ml,cmp ); // Сортування списку
        System.out.println(ml); // Відображення списку
    }
}
```



Використання алгоритмів

Програма: використання алгоритмів – результат

```
[12, 5, 17, 8, 21, 25, 9]
[5, 8, 9, 12, 17, 21, 25]
[25, 21, 17, 12, 9, 8, 5]
[25, 21, 100, 17, 200, 12, 300, 9, 8, 5]
[300, 200, 100, 25, 21, 17, 12, 9, 8, 5]
[ПЕРШИЙ-12, ДРУГИЙ-5, ТРЕТІЙ-17, ЧЕТВЕРТИЙ-8, П'ЯТИЙ-21, ШОСТИЙ-25, СЬОМИЙ-9]
Найбільше значення: ШОСТИЙ-25
Найменше значення: ДРУГИЙ-5
[ДРУГИЙ-5, ЧЕТВЕРТИЙ-8, СЬОМИЙ-9, ПЕРШИЙ-12, ТРЕТІЙ-17, П'ЯТИЙ-21, ШОСТИЙ-25]
```



Додаткові відомості. Клас Stack

- Підклас класу Vector
- Реалізує стандартний стек за формою LIFO (Last In First Out)

Додаткові (крім Vector) методи класу Stack

Метод	Описання
<code>boolean empty()</code>	Повертає <code>true</code> якщо стек порожній, і <code>false</code> якщо стек містить елементи
<code>Object peek()</code>	Повертає елемент на вершині стека (але не видаляє цей елемент!)
<code>Object pop()</code>	Повертає елемент, що знаходиться на вершині стека. При цьому елемент зі стеку видаляється
<code>Object push(Object obj)</code>	Поміщає елемент у стек (у вершину стека) та повертає посилання на нього
<code>int search(Object obj)</code>	Виконує пошук елемента, переданого аргументом методу, в стеку, з якого викликається метод. Якщо елемент знайдено, повертається його зміщення від вершини стека. Якщо елемент не знайдено, повертається значення <code>-1</code>



Використання стека

```
import java.util.Stack;
import java.util.Random;
class Demo{
    public static void main(String[] args){
        // Для генерування випадкових чисел:
        Random rnd= new Random();
        // Порожній стек:
        Stack<Integer> st=new Stack<>();
        if(st.empty()){
            // Заповнення стека:
            for(int k=1;k<=5;k++){
                System.out.println("Додано "+k+"-й елемент: "+
                    st.push(rnd.nextInt(101)));
            }
        }
        // Вміст стеку:
        System.out.println("Сформовано стек: "+st);
        // Вилучення елементів:
        while(!st.empty()){
            System.out.println("Прочитано елемент: "+st.pop());
            System.out.println("Вміст стеку: "+st);
        }
    }
}
```



Використання стека

Програма: використання стека – результат

```
Додано 1-й елемент: 85
Додано 2-й елемент: 84
Додано 3-й елемент: 21
Додано 4-й елемент: 61
Додано 5-й елемент: 82
Сформовано стек: [85, 84, 21, 61, 82]
Прочитано елемент: 82
Вміст стеку: [85, 84, 21, 61]
Прочитано елемент: 61
Вміст стеку: [85, 84, 21]
Прочитаний елемент: 21
Вміст стеку: [85, 84]
Прочитано елемент: 84
Вміст стеку: [85]
Прочитано елемент: 85
Вміст стеку: []
```

Додаткові відомості. Клас Hashtable

- Успадковує абстрактний клас Dictionary і реалізує інтерфейс Map
- Зберігає пари ключ/значення у хеш-таблиці

Конструктор	Описання
<code>Hashtable()</code>	Створюється пуста таблиця
<code>Hashtable(int size)</code>	Створення таблиці заданого розміру
<code>Hashtable(int size, float fillRatio)</code>	Створення таблиці заданого розміру та часток заповнення (за замовчуванням 0.75)
<code>Hashtable(Map m)</code>	Створюється таблиця, що ініціалізується картою

Деякі методи класу Hashtable

Метод	Описання
<code>clear()</code>	Очищення таблиці
<code>Object get(Object key)</code>	Повертає елемент за ключем
<code>Object put(Object k, Object v)</code>	Додає елемент із ключем до таблиці
<code>Object remove(Object key)</code>	Видаляє елемент за ключем
<code>int size()</code>	Розмір таблиці



Використання Hashtable

```
import java.util.Hashtable;
class Demo{
    public static void main(String[] args){
        // Пуста таблиця:
        Hashtable<String,Integer> ht=new Hashtable<>();
        // Додавання елементів до таблиці:
        ht.put("Іван Іваненко",35);
        ht.put("Петро Петренко",27);
        ht.put("Сидор Сидоренко",43);
        // Вміст таблиці:
        System.out.println("Вміст таблиці: "+ht);
        // Запит елемента за ключем:
        String name="Сидор Сидоренко";
        // String name="сидор сидоренко";
        System.out.println("Користувач: "+name);
        if(ht.containsKey(name))
            System.out.println("Вік: "+ht.get(name));
        else
            System.out.println("Такого користувача немає");
    }
}
```

Результат

Вміст таблиці: {Іван Іваненко=35, Петро Петренко=27, Сидор Сидоренко=43}
Користувач: Сидор Сидоренко
Вік: 43

Дякую за увагу!

Запитання?